Cross-Border Solution Architecture Playbook

How I would approach taking enterprise and BFSI clients from first call to scale, across languages, cultures and time zones.

Discover → Design → Integrate → Launch → Optimize

One architect, one solution, end to end.

Solution architecture & integration

End-to-end solutions and workflows, from discovery to post-go-live.

Pre-sales & technical discovery

Discovery sessions, product deep-dives, security queries and POCs with Sales.

Enterprise & BFSI delivery

Trusted advisor through strict security, compliance and data privacy constraints.

Cross-functional bridge

Market needs and client feedback into the product roadmap.

Troubleshooting & optimization

Fix integration issues, tune API and SDK performance, build execution frameworks.

Delivery flow

1

🔍 Discovery: know the client and the region

Regional nuance · communication styles

× A “yes” on the call can mean “I heard you”, not “we agree”.

✓ Map stakeholders, systems and constraints, plus language, time overlap and data law. Meet the local partner.

2

📝 Requirements: agree the scope and the words

Terminology gaps · translation teams

× "Webhook" translated three ways in three documents.

✓ A signed-off scope and one shared glossary. AI drafts translations, a native-speaking partner reviews them.

3

🗺️ Solution design: a visual blueprint

Visual integration blueprints

× Long English documents nobody finishes reading.

✓ Diagrams and code first, short sentences second. See the live demo below.

4

🔌 Integration: an API kit that runs

Standardized API docs · Postman

× Every team reads the API docs differently.

✓ A shared Postman template, filled with the real endpoints during integration. Engineers test instead of reading.

5

🔄 Delivery cadence: async first, live in the overlap

Async loops · time zone friction · high-context cultures

× One question takes three days of email. Blockers are never said in the group call.

✓ Short calls in shared hours, a written recap in both languages, and a 1:1 with the tech lead.

6

🛡️ Security, compliance and go-live

Local data privacy mandates

× Go-live stalls while security asks where the selfies are stored.

✓ Bring a data map, the local rules and the security documents to the first security call.

7 📈 Post-go-live: hypercare, scale and support

Troubleshooting · optimization · long-term adoption

× After launch, drop-offs and slow responses go unnoticed until customers complain.

✓ Hypercare, live dashboards with alerts, and a named escalation path in the client's language.

Proposed targets · to agree with the client in the SLA

🩺 **Hypercare** PROPOSED

First 30 days: daily check-in, then weekly. A shared issue log.

📊 **Monitor** PROPOSED

Completion funnel, p95 latency, error codes and auto-approve rate, with alerts on drops.

🚀 **Scale** PROPOSED

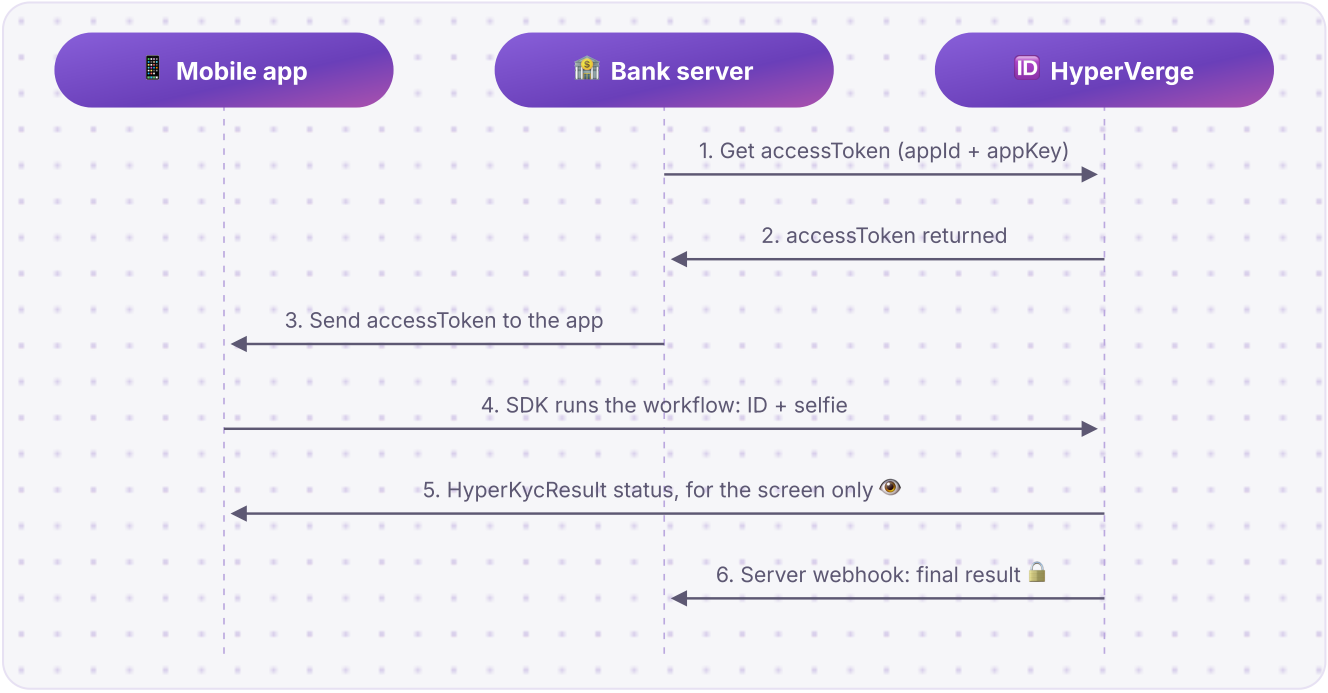
Load-test at 3× peak before campaigns, paydays, Tết and Ramadan.

🆘 **Support** PROPOSED

P1 response in 30 min, root cause in 48 h, planned SDK upgrades.

Live demo: the blueprint, in their language LIVE

MESSAGE FLOW



👁 Display only, never used to approve 🔒 Server-to-server: the only result the bank decides on

Arabic mirrors right-to-left

API kit

Postman collection SAMPLE

Four requests in HyperKYC SDK terms: appId/appKey, accessToken, workflowId and transactionId.

- POST **Generate accessToken** — server only, never in the app
- POST **Fetch results by transactionId** — for missed webhooks
- POST **Replay a sample webhook** — to your server
- POST **Replay it again** — duplicates must be safe

A sample collection to import into Postman. Endpoint URLs and the webhook body are placeholders to replace from your HyperVerge docs.

Key best practices

INTEGRATION

Keys stay on the server

Generate the accessToken server-side. The SDK's own README calls appId/appKey in the app “less secure”.

Decide on the server, never the app

Approve from the webhook or results API. The HyperKycResult status is for the screen.

Make webhooks idempotent

The same event can arrive twice. Process it once, by transaction ID.

Test in real conditions

Low-end Android, weak networks, low light, every local ID card version.

COMMUNICATION

Write it down, in both languages

Every call ends with a recap: who does what, by when.

One glossary, everywhere

Lock terms before translating. A native-speaking partner reviews.

Meet in the overlap

Short calls in shared hours to screen-share and debug. Everything else async.

Data stays where the law says

In-country region, a reference ID instead of images, agreed retention.

Regions I plan for

Vietnam
Vietnamese
IST +1:30

Indonesia
Bahasa Indonesia
IST +1:30

Philippines
Filipino, English
IST +2:30

Japan
Japanese
IST +3:30

Saudi Arabia
Arabic
IST -2:30

UAE
Arabic, English
IST -1:30

Senegal · Côte d'Ivoire
French
IST -5:30

Brazil
Portuguese
IST -8:30

Language and time difference from Bangalore. Click a region to see the demo in its language. Patterns to plan for, checked with the local partner in week one.

Proposed solution for a fictional client, Sen Bank; not a record of past KYC delivery. SDK terms and status values follow HyperVerge's public Flutter SDK (hyperkyc_flutter on pub.dev); endpoint URLs and the webhook payload are placeholders. Translations are AI-drafted and not yet reviewed by a native speaker. Independent work sample styled after HyperVerge's public site; not an official HyperVerge page.